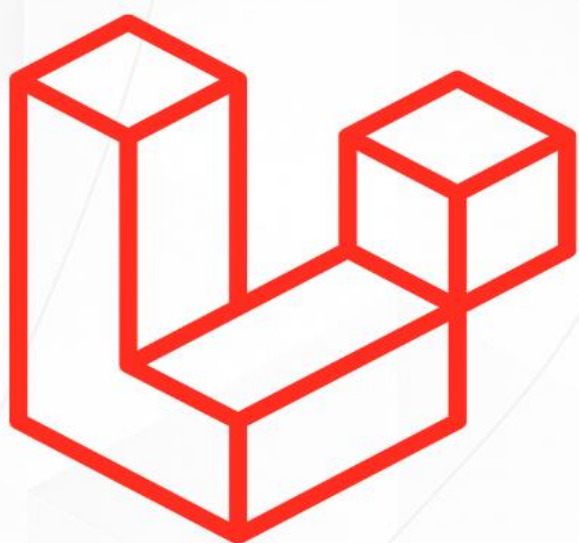




Laravel

Essencial

Do básico à estrutura, API, conceitos e **projetos reais** com boas práticas.



SatellaSoft
TECNOLOGIA EM DADOS DA INFORMAÇÃO

Por: **Gunnar Correa**



Sumário

Direitos Autorais.....	3
Sites úteis:.....	3
Material do curso.....	3
Sobre o curso.....	4
Sobre o autor	5
Grupo de estudos no Whatsapp.....	6
O que é o Laravel.....	7
Como instalar o Composer	8
Instalação no Windows.....	8
Instalação no Linux.....	8
O que é End of Life (EOL)?	9
Criando um novo projeto Laravel.....	10
Visual Studio Code (VS Code)	11
Extensões recomendadas para Laravel no VS Code	12
Estrutura de pastas do Laravel.....	13
Boas Práticas.....	14
Ciclo de Vida HTTP no Laravel	15
Padrão MVC.....	16
O que são verbos HTTP?	17
Rotas.....	18
Rotas nomeadas.....	18
Criando uma Controller.....	19
Camada de Visão (View).....	20
API Game	22
Softwares utilizados	22
Comandos utilizados.....	22

Endpoints da API.....	22
-----------------------	----

Direitos Autorais

Este material é de propriedade exclusiva da **SatellaSoft.com**, desenvolvido por **Gunnar Corrêa**. Foi originalmente criado para atender aos cursos da **Udemy** e da própria **SatellaSoft**, sendo destinado exclusivamente aos alunos do curso de **Laravel**.

Fica expressamente **proibida a reprodução, distribuição, compartilhamento ou comercialização**, total ou parcial, por qualquer meio, sem a prévia e formal autorização do autor. O descumprimento poderá acarretar sanções civis e criminais, conforme a legislação vigente.

Sites úteis:

- <https://satellasoft.com> – Artigos de tecnologia
- [Academy.satellasoft.com](https://academy.satellasoft.com) – Cursos de tecnologia
- [Suiteensinos.com.br](https://suiteensinos.com.br) – Cursos utilitários

Material do curso

Todo material, código e recursos utilizados durante a aula encontram-se em nosso repositório oficial.

Confira em: <https://github.com/satellasoft/treinamento-laravel>

Sobre o curso

Este curso foi criado para quem quer aprender Laravel de forma direta, profissional e aplicável ao mercado real.

Vamos construir juntos um projeto completo: uma API com operações CRUD e um site que consome esses dados usando Blade. Durante as aulas, você vai entender toda a estrutura do Laravel, o padrão MVC, como trabalhar com migrations, models, rotas, controllers, validação de dados, API Resource e organização de código.

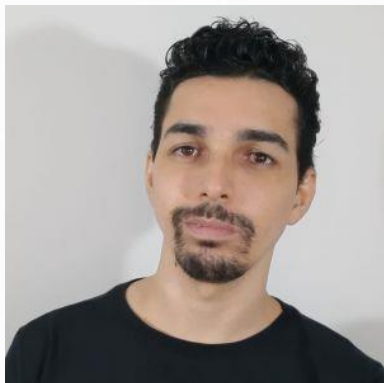
O objetivo é que você não apenas "copie e cole", mas entenda como pensar uma aplicação desde a base, aplicando boas práticas e preparando seu código para escalar.

Ideal para quem deseja sair do básico e aprender a criar projetos reais com Laravel, dominando a estrutura e os conceitos mais utilizados em aplicações modernas.

Ao final, você terá um projeto funcional e pronto para servir como base em aplicações maiores. Esse é o Laravel de verdade, aplicado na prática.

Além disso, este curso é voltado para quem busca entender como estruturar aplicações escaláveis, com separação clara de responsabilidades, e quer ganhar confiança para trabalhar com frameworks modernos em ambientes profissionais, seja para freelas, startups ou grandes times.

Sobre o autor



Gunnar Corrêa é Arquiteto de Software, empreendedor e educador com ampla experiência no desenvolvimento de soluções digitais e formação de profissionais na área de tecnologia. Fundador da SatellaSoft, atua com foco em alta performance, escalabilidade e educação técnica de qualidade. Ao longo dos anos, tem ajudado empresas e pessoas a evoluírem por meio de conteúdos práticos, diretos e aplicáveis ao mercado real.

Redes sociais



Grupo de estudos no Whatsapp

Temos um grupo exclusivo no WhatsApp para alunos do curso de Laravel na Udemy.

Apenas alunos têm acesso, e o foco são dúvidas, suporte e discussões sobre os projetos desenvolvidos ao longo do curso.

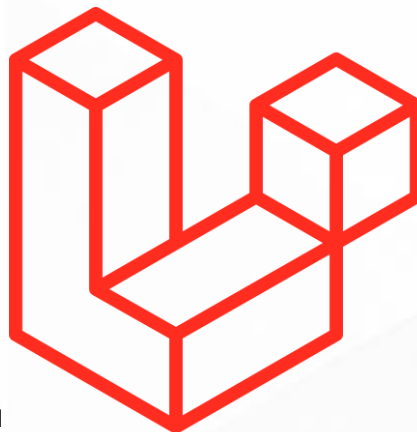
Participe em: <https://chat.whatsapp.com/FZ7sISGkzeeFB5CTyZSg8O>



Laravel
Essencial

O que é o Laravel

O **Laravel** é um framework PHP moderno, robusto e elegante, utilizado para o desenvolvimento de aplicações web. Ele foi criado para tornar o desenvolvimento mais simples, organizado e eficiente, oferecendo uma sintaxe expressiva e recursos avançados que ajudam os desenvolvedores a entregar projetos de alta qualidade em menos tempo.



Laravel segue o padrão arquitetural **MVC (Model-View-Controller)**, que organiza o código em camadas bem definidas, facilitando a manutenção, escalabilidade e legibilidade dos projetos.

Além disso, o framework oferece uma série de ferramentas integradas, como sistema de roteamento, autenticação, envio de e-mails, filas, jobs, cache, testes automatizados, integração com banco de dados, além de um poderoso sistema de migrations e seeders. Tudo isso contribui para que os desenvolvedores possam focar no que realmente importa: a regra de negócio e a entrega de valor ao cliente.

Outro grande diferencial do Laravel é a sua vasta comunidade e seu ecossistema, que inclui ferramentas como o **Laravel Breeze**, **Jetstream**, **Laravel Sanctum**, **Laravel Passport**, **Laravel Horizon**, entre outras. Esses pacotes aceleram ainda mais o desenvolvimento e oferecem soluções prontas e seguras para diversos desafios do dia a dia.

Quando falamos em **Laravel**, falamos em **produtividade e segurança**. É uma ferramenta que permite criar aplicações modernas, robustas e confiáveis, com agilidade e alto padrão de qualidade.

Site oficial: <https://laravel.com>

Como instalar o Composer

O **Composer** é um gerenciador de dependências para PHP, essencial para trabalhar com projetos modernos, como o Laravel. Com ele, você instala e gerencia bibliotecas e frameworks de forma simples e eficiente.

Instalação no Windows

1. Acesse o site oficial: <https://getcomposer.org>
2. Baixe o instalador: **Composer-Setup.exe**
3. Execute o arquivo e siga os passos do assistente.
4. Finalize a instalação e, no terminal (Prompt, PowerShell ou Git Bash), digite: **composer -version**

Instalação no Linux

O **Composer** é um gerenciador de dependências para PHP, indispensável para trabalhar com frameworks como o Laravel. Sua instalação no Linux é simples e rápida. No terminal, execute:

```
cd ~  
  
php -r "copy('https://getcomposer.org/installer',  
'composer-setup.php');"  
  
php composer-setup.php --install-dir=/usr/local/bin --  
filename=composer  
  
php -r "unlink('composer-setup.php');"
```

Teste:

```
composer -version
```


O que é End of Life (EOL)?

End of Life (EOL) significa que uma determinada versão de um software não receberá mais **atualizações, melhorias ou correções de segurança**. No caso do Laravel e de qualquer outra tecnologia, utilizar versões fora do EOL representa um risco, pois o sistema fica exposto a vulnerabilidades e incompatibilidades.

Por isso, é fundamental acompanhar o ciclo de vida das versões e manter seus projetos sempre atualizados dentro de versões que ainda recebem **suporte e segurança**.

Quando falamos em EOL, falamos diretamente em **risco, segurança e estabilidade**.

Leia mais em: <https://laravel.com/docs/master/releases>

Version	PHP (*)	Release	Bug Fixes Until	Security Fixes Until
9	8.0 - 8.2	February 8th, 2022	August 8th, 2023	February 6th, 2024
10	8.1 - 8.3	February 14th, 2023	August 6th, 2024	February 4th, 2025
11	8.2 - 8.4	March 12th, 2024	September 3rd, 2025	March 12th, 2026
12	8.2 - 8.4	February 24th, 2025	August 13th, 2026	February 24th, 2027

Figura 1: Reprodução site Laravel

Criando um novo projeto Laravel

Com o **Composer** instalado, criar um projeto Laravel é simples. No terminal, execute:

```
composer create-project laravel/laravel nome-do-projeto
```

Substitua `nome-do-projeto` pelo nome que desejar para sua aplicação.

Aguarde a instalação e, ao finalizar, acesse a pasta do projeto:

```
cd nome-do-projeto
```

Pronto! Seu projeto Laravel está criado e pronto para desenvolvimento.

Visual Studio Code (VS Code)

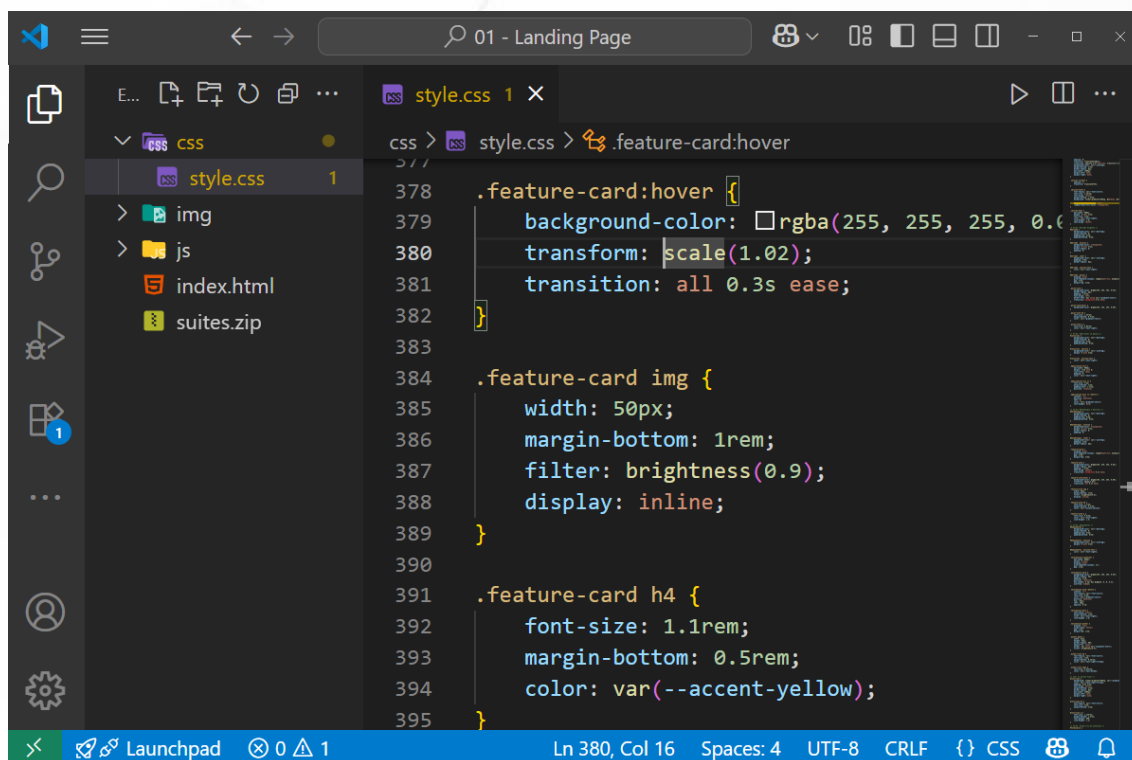
O **Visual Studio Code**, ou simplesmente **VS Code**, é um editor de código leve, gratuito e multiplataforma, desenvolvido pela **Microsoft**. Ele é extremamente popular entre desenvolvedores por oferecer uma interface limpa, alta performance e suporte a diversas linguagens, incluindo PHP e Laravel.



O VS Code possui uma grande variedade de extensões que facilitam o desenvolvimento, como formatação automática, autocomplete, lint, integração com Git e terminais embutidos.

Quando falamos em desenvolvimento Laravel, o VS Code se torna uma escolha natural pela sua praticidade, personalização e produtividade no dia a dia.

Site oficial: <https://code.visualstudio.com/>

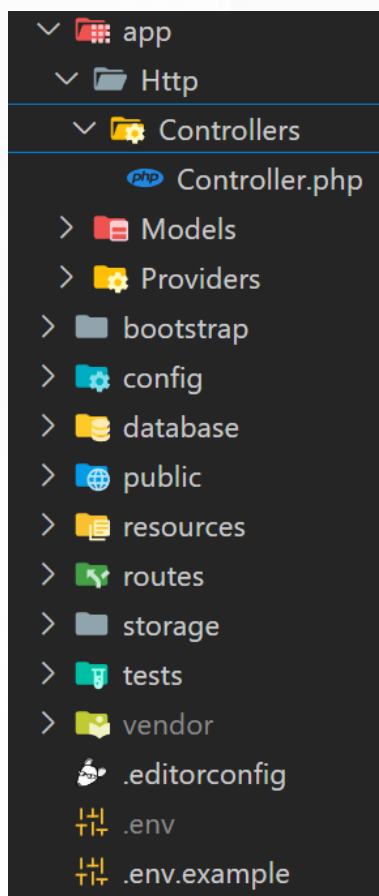


Extensões recomendadas para Laravel no VS Code

Para tornar seu desenvolvimento mais produtivo, essas são algumas extensões essenciais:

- **PHP Intelephense** — Autocomplete, IntelliSense e suporte inteligente para PHP.
- **Laravel Blade Snippets** — Suporte e snippets para arquivos Blade (.blade.php).
- **Laravel Artisan** — Permite executar comandos Artisan diretamente no VS Code.
- **Laravel Extra Intellisense** — Melhora o autocomplete, reconhecendo facades, helpers e métodos mágicos do Laravel.
- **PHP Namespace Resolver** — Organiza e importa automaticamente os namespaces.
- **DotENV** — Suporte e destaque de sintaxe para arquivos .env.
- **GitLens** — Potencializa o uso do Git dentro do VS Code.
- **Prettier** ou **PHP CS Fixer** — Para formatação automática de código.

Estrutura de pastas do Laravel



Ao criar um projeto Laravel, você verá uma estrutura de pastas organizada, cada uma com sua responsabilidade.

Estrutura principal:

- **app/** → Código principal da aplicação (Models, Services, Providers, etc.).
- **bootstrap/** → Arquivos de bootstrapping. Inicialização da aplicação.
- **config/** → Arquivos de configuração do sistema.
- **database/** → Migrations, seeders e factories.
- **public/** → Pasta pública (index.php, imagens, CSS, JS).
- **resources/** → Views (Blade), assets e arquivos de tradução.
- **routes/** → Arquivos de rotas (web.php, api.php, console.php).
- **storage/** → Logs, cache e arquivos gerados pela aplicação.
- **tests/** → Testes unitários e de feature.
- **vendor/** → Dependências gerenciadas pelo Composer.

Cada uma dessas pastas tem um papel essencial na organização e funcionamento da aplicação, seguindo os padrões do framework e garantindo escalabilidade no desenvolvimento.

Boas Práticas

Adotar boas práticas no Laravel garante um código mais limpo, organizado, escalável e fácil de manter. Aqui estão algumas recomendações fundamentais:

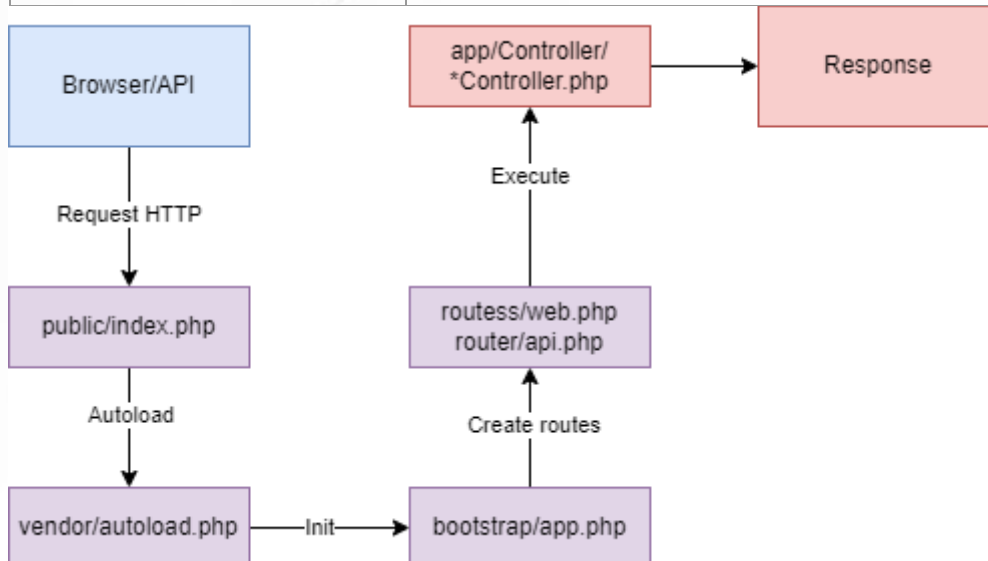
- **Siga o padrão MVC:** Mantenha separadas as responsabilidades entre Models, Views e Controllers.
- **Use migrations:** Nunca crie tabelas manualmente. Use migrations para versionar seu banco de dados.
- **Valide dados no Controller ou via Form Request:** Nunca confie em dados vindos do usuário sem validação.
- **Utilize Eloquent de forma inteligente:** Prefira relacionamentos (hasMany, belongsTo) e Query Scopes para consultas mais limpas.
- **Evite lógica no Controller:** Crie Services, Helpers ou Actions para concentrar regras de negócio.
- **Nomes claros:** Dê nomes claros para métodos, variáveis, rotas e arquivos.
- **Aproveite o poder dos Resources:** Use API Resource para transformar dados antes de enviar para o front.
- **Utilize o .env:** Nunca exponha dados sensíveis no código. Guarde no arquivo .env.
- **Gerencie dependências com Composer:** Atualize e organize suas dependências corretamente.
- **Teste sua aplicação:** Utilize php artisan test para criar testes e garantir a estabilidade do seu sistema.

Manter essas práticas torna seu projeto mais profissional, seguro e preparado para crescer.

Ciclo de Vida HTTP no Laravel

A partir do **Laravel 11**, e também no **Laravel 12**, o framework passou por uma importante reformulação na sua estrutura interna, tornando-se ainda mais simples e direto. O arquivo **App \ Http \ Kernel.php** deixou de existir, e o gerenciamento de middlewares, providers e rotas agora é feito diretamente no arquivo **bootstrap/app.php**.

Arquivo/Classe	Função Principal
public/index.php	Ponto de entrada. Recebe a requisição e inicializa o framework.
vendor/autoload.php	Carrega as dependências via Composer.
bootstrap/app.php	Cria a aplicação, registra middlewares, providers e rotas.
routes/web.php/ api.php	Define as rotas que serão usadas na aplicação.
App/Http/Controllers/	Controllers que processam a lógica da requisição e retornam a resposta.
resources/views/	Views Blade (quando usado).



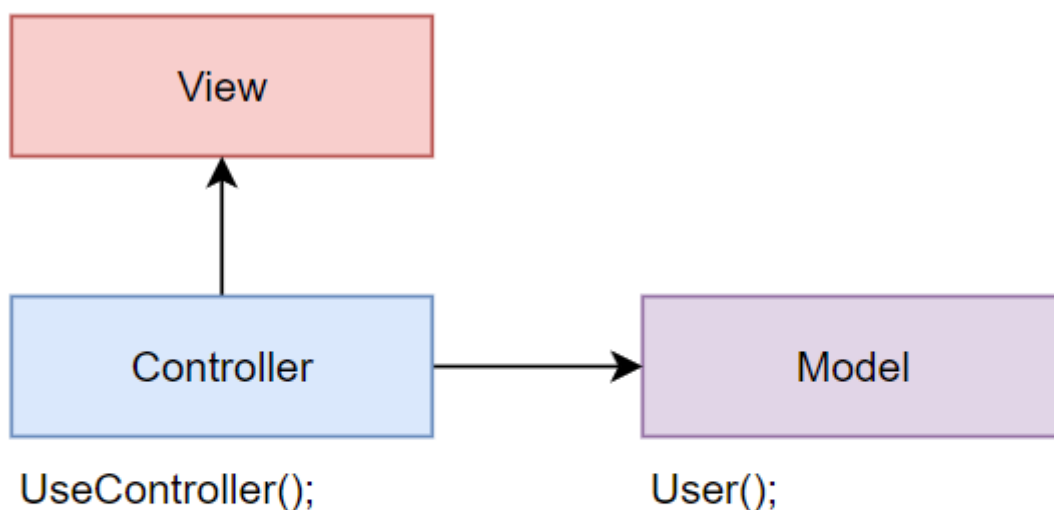
Padrão MVC

O **MVC** é um padrão de arquitetura que separa a aplicação em três camadas:

- **Model:** cuida dos dados e da regra de negócio.
- **View:** é a interface que exibe as informações para o usuário.
- **Controller:** recebe as requisições, processa e conecta o Model à View.

No Laravel, o uso do MVC deixa o código mais organizado, limpo e fácil de manter.

index.blade.php



```
php artisan make:controller UserController
```

```
php artisan make:model User
```


O que são verbos HTTP?

Os **verbos HTTP** definem a ação que queremos realizar em uma requisição. Eles indicam se vamos **buscar, criar, atualizar ou excluir** informações em uma API ou servidor.

Os principais são:

- **GET:** buscar dados.
- **POST:** criar novos dados.
- **PUT / PATCH:** atualizar dados.
- **DELETE:** remover dados.

Cada verbo tem um propósito e é essencial para o funcionamento correto de APIs e aplicações web.

Ferramentas sugeridas:

- <https://insomnia.rest/>
- <https://www.postman.com/>

GET**GET** Buscar ou listar informações.**POST****POST** Criar um novo recurso.**PUT****PUT** Atualizar um recurso (substituição total).**PATCH****PATCH** Atualizar parcialmente um recurso.**DELETE****DELETE** Remover um recurso.

Rotas

As **rotas** são responsáveis por definir quais URLs a aplicação responde e qual ação será executada. No Laravel, as rotas são definidas nos arquivos **routes/web.php** (para web) ou **routes/api.php** (para APIs).

```
Route::get('/sobre', function () {  
    return 'Sobre nós';  
});
```

Essa rota responde ao endereço /sobre usando o verbo **GET**.

Rotas nomeadas

As **rotas nomeadas** facilitam a geração de URLs e redirecionamentos. Em vez de usar a URL diretamente, você usa o nome da rota.

```
Route::get('/dashboard', [DashboardController::class, 'index'])  
    ->name('dashboard');
```

Usando a rota nomeada no código:

```
return redirect()->route('dashboard');
```

Criando uma Controller

No terminal, execute:

```
php artisan make:controller ProdutoController
```

Isso cria o arquivo:

```
app/Http/Controllers/ProdutoController.php
```

Exemplo simples de Controller:

```
<?php
namespace App\Http\Controllers;
use Illuminate\Http\Request;
class ProdutoController extends Controller
{
    public function index()
    {
        return 'Listando produtos';
    }
}
```

Definindo a rota:

```
Route::get('/produtos', [ProdutoController::class, 'index']);
```

Acessando /produtos, você verá:

```
Listando produtos
```

Camada de Visão (View)

A **camada de visão** é responsável por exibir as informações para o usuário. No Laravel, isso é feito através dos arquivos **Blade**, que ficam em **resources/views**.

Ela não contém regras de negócio, apenas organiza o que será mostrado na tela, como textos, listas, tabelas e formulários.

A View recebe dados da Controller e monta a interface de forma dinâmica.

```
<?php
namespace App\Http\Controllers;
class ProdutoController extends Controller
{
    public function index()
    {
        $produtos = ['Mouse', 'Teclado', 'Monitor'];
        return view('produtos.index', compact('produtos'));
    }
}
```

Rota:

```
Route::get('/produtos', [ProdutoController::class, 'index']);
```

View Blade (resources/views/produtos/index.blade.php):

```
<!DOCTYPE html>

<html>

<head>

  <title>Produtos</title>

</head>

<body>

  <h1>Lista de Produtos</h1>

  <ul>

    @foreach($produtos as $produto)

      <li>{{ $produto }}</li>

    @endforeach

  </ul>

</body>

</html>
```

API Game

Vamos desenvolver agora um CRUD completo de games por meio de APIs. Todo o código encontra-se no repositório oficial.

Softwares utilizados

- XAMPP - https://www.apachefriends.org/pt_br/index.html
- Insomnia Rest - <https://insomnia.rest>
- Veja como instalar - <https://satellasoft.com/artigo/php/como-baixar-e-instalar-o-xampp-no-windows>

Comandos utilizados

- **php artisan make:model Game** – Cria uma model
- **php artisan make:controller GameController** – Cria a controladora
- **php artisan make:migration create_games_table** – Cria uma Migration
- **php artisan migrate** – Roda a Migration

Endpoints da API

Método	Endpoint	Descrição
GET	/api/games	Listar todos os jogos
POST	/api/games	Cadastrar um novo jogo
GET	/api/games/{id}	Buscar detalhes de um jogo
PUT	/api/games/{id}	Atualizar um jogo (total)
PATCH	/api/games/{id}	Atualizar um jogo (parcial)
DELETE	/api/games/{id}	Remover um jogo

```
1 Route::prefix('users')->group(function () {  
2     Route::get('/', [UserController::class, 'index']);  
3     Route::get('/{id}', [UserController::class, 'show']);  
4     Route::post('/', [UserController::class, 'store']);  
5     Route::put('/{id}', [UserController::class, 'update']);  
6     Route::patch('/{id}', [UserController::class, 'update']);  
7     Route::delete('/{id}', [UserController::class, 'destroy']);  
8 });
```

Documentação: <https://laravel.com/docs/12.x/routing>

Exemplo de rotas para um caso mais real de gerenciamento de usuários (extraído de uma aplicação real).

Método	Rota	Controller	Status Esperado
GET	/users	index()	200 OK
GET	/users/{id}	show()	200 OK, 404 Not Found
POST	/users	store()	201 Created, 422 Unprocessable Entity
PUT	/users/{id}	update()	200 OK, 404, 422
PATCH	/users/{id}	update()	200 OK, 404, 422
DELETE	/users/{id}	destroy()	204 No Content, 404

Http Status Code: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Status>

Validação: <https://laravel.com/docs/11.x/validation>

SatellaSoft Cookies

GET http://localhost:8000/api/games/ Send 200 OK 176 ms 768 B

Filter

Api Game

- DEL Delete
- GET Show
- GET Index
- PUT Update
- POST Store
- GET Ping

Body Auth Query Headers Docs

Preview Headers Cookies Timeline

```
1 {
2   "message": "List of games",
3   "data": [
4     {
5       "id": 1,
6       "name": "Dino Crisis 2",
7       "description": "Dino",
8       "thumb": null,
9       "video": null,
10      "release_date": "2000-09-01",
11      "created_at": "2025-06-26T00:15:18.000000Z",
12      "updated_at": "2025-06-26T00:15:18.000000Z"
13    },
14    {
15      "id": 3,
16      "name": "Resident Evil 3 - Nemesis",
17      "description": "Resident Evil 3: Nemesis, conhecido
18      acontecimentos de Resident Evil 2.",
19      "thumb": null,
20      "video": null,
21      "release_date": "1999-09-22",
22      "created_at": "2025-06-26T00:25:08.000000Z",
23      "updated_at": "2025-06-26T00:50:59.000000Z"
24    }
25  ]
26 }
```


ACELERE SUA CARREIRA EM TECNOLOGIA

Mentoria personalizada para desenvolvedores que buscam
excelência técnica e crescimento profissional

[Conhecer](#)

Mentoria Personalizada

Planos sob medida focados em seus
objetivos específicos



Sessões Online

Encontros práticos via Google Meet com
resultados concretos



Acompanhamento

Monitoramento contínuo do seu progresso
e evolução



Resultados Rápidos

Metodologia comprovada para acelerar seu
crescimento

Expertise Comprovada

Mais de 15 anos de experiência no mercado. Fundadores da SatellaSoft e
criadores de plataformas reconhecidas como SatellaSoft Academy e
SuiteEnsinos.

[Laravel Expert](#)[AWS Solutions](#)[DevOps](#)[Arquitetura](#)[Consultoria](#)